

Article

Practical Comparison Between the CI/CD Platforms Azure DevOps and GitHub

Vladislav Manolov, Daniela Gotseva and Nikolay Hinov * 

Department of Computer Systems, Faculty of Computer Systems and Technologies, Technical University of Sofia, 1000 Sofia, Bulgaria; vladislaviliyanovmanolov@gmail.com (V.M.); dgoceva@tu-sofia.bg (D.G.)

* Correspondence: hinov@tu-sofia.bg

Abstract: Continuous integration and delivery are essential for modern software development, enabling teams to automate testing, streamline deployments, and deliver high-quality software more efficiently. As DevOps adoption grows, selecting the right CI/CD platform is essential for optimizing workflows. Azure DevOps and GitHub, both under Microsoft, are leading solutions with distinct features and target audiences. This paper compares Azure DevOps and GitHub, evaluating their CI/CD capabilities, scalability, security, pricing, and usability. It explores their integration with cloud environments, automation workflows, and suitability for teams of varying sizes. Security features, including access controls, vulnerability scanning, and compliance, are analyzed to assess their suitability for organizational needs. Cost-effectiveness is also examined through licensing models and total ownership costs. This study leverages real-world case studies and industry trends to guide organizations in selecting the right CI/CD tools. Whether seeking a fully managed DevOps suite or a flexible, Git-native platform, understanding the strengths and limitations of Azure DevOps and GitHub is crucial for optimizing development and meeting long-term scalability goals.

Keywords: Azure DevOps; CI/CD; cloud; GitHub; DevOps automation; version control



Academic Editors: Stefano Rinaldi and Alan Oliveira De Sá

Received: 7 March 2025

Revised: 24 March 2025

Accepted: 28 March 2025

Published: 31 March 2025

Citation: Manolov, V.; Gotseva, D.; Hinov, N. Practical Comparison Between the CI/CD Platforms Azure DevOps and GitHub. *Future Internet* **2025**, *17*, 153. <https://doi.org/10.3390/fi17040153>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The landscape of software development has undergone a significant transformation in recent years, driven by the increasing complexity of applications, the demand for rapid innovation, and the widespread adoption of cloud computing. Traditional software development methodologies, which relied on manual integration, lengthy release cycles, and infrequent deployments, have failed to meet the growing expectations for speed, reliability, and scalability. As a result, DevOps has emerged as a fundamental change in thinking, bridging the gap between development and operations teams by fostering a culture of automation, continuous improvement, and cross-functional collaboration. At the heart of DevOps is continuous integration and continuous deployment (CI/CD), which enables organizations to automate software building, testing, and deployment processes. By incorporating CI/CD pipelines into their workflows, software teams can reduce human error, improve software quality, and accelerate time-to-market [1,2].

Continuous integration ensures that code changes are automatically evaluated and merged into a shared repository, preventing integration conflicts and guaranteeing a stable codebase. Continuous deployment extends this process by automating software release into production environments, allowing for seamless and frequent updates. These practices have become essential in modern software engineering, particularly in industries where

reliability and speed are critical, such as finance, healthcare, and e-commerce. Organizations that fail to implement CI/CD risk falling behind their competitors, which can innovate faster and respond more efficiently to market demands. Given the importance of CI/CD in streamlining software delivery, selecting the right platform is a strategic decision that can significantly impact a development team's productivity, operational efficiency, and overall software lifecycle management.

Among the various CI/CD tools available today, Azure DevOps and GitHub have emerged as two of the most widely used platforms, offering robust solutions for source code management, workflow automation, and software deployment. This comparison is based on insights from Fortune Global 500 companies. Although Microsoft owns both platforms, they serve distinct audiences and have evolved to support different development workflows. Azure DevOps, formerly known as Visual Studio Team Services (VSTS), is a comprehensive DevOps suite that provides an end-to-end solution for enterprise software development. It includes version control, build automation, testing, deployment, and project management services. It is particularly well suited for large-scale software teams and enterprises that require deep integration with the Microsoft ecosystem, particularly Azure. GitHub, initially created as a platform for hosting and collaborating on Git repositories, has expanded its capabilities by introducing GitHub Actions—a flexible and scalable CI/CD automation solution. GitHub is widely recognized for its developer-friendly approach, dominance in the open-source community, and seamless integration with Git-based workflows. Azure DevOps, designed as a structured and comprehensive DevOps solution, offers a more lightweight and flexible approach, appealing to individual developers, startups, and teams prioritizing Git-native automation [2–4].

Despite their similarities in providing CI/CD automation, the differences between Azure DevOps and GitHub can have a significant impact on an organization's development processes. Azure DevOps provides a comprehensive suite of DevOps services, including Azure Pipelines for continuous integration and continuous deployment (CI/CD), Azure Repos for version control, Azure Boards for project tracking, Azure Test Plans for testing, and Azure Artifacts for package management. This makes it an all-in-one solution for enterprises that require a structured approach to DevOps, particularly those that rely on Microsoft's cloud services. On the other hand, GitHub takes a more modular approach, allowing developers to integrate third-party CI/CD tools alongside GitHub Actions. Its emphasis on community-driven development, ease of collaboration, and an intuitive user interface has made it the preferred choice for open-source contributors and agile development teams.

Choosing between Azure DevOps and GitHub depends on several key factors, including the project's complexity, the size of the development team, the level of automation required, security considerations, and budget constraints. Enterprises that require strict governance, advanced security features, and deep integration with Microsoft tools may find Azure DevOps to be the ideal solution. Conversely, teams prioritizing flexibility, open-source collaboration, and Git-native automation may benefit more from GitHub's streamlined approach. The cost structure of each platform also plays a crucial role in decision-making, as organizations must consider licensing models, subscription plans, and the total cost of ownership when selecting a CI/CD tool [5].

This paper compares Azure DevOps and GitHub by evaluating their features, scalability, security, pricing, and overall usability. By analyzing real-world case studies, industry best practices, and adoption trends, this study offers valuable insights to software engineers, DevOps professionals, and IT decision-makers evaluating these platforms for their development pipelines. Understanding the strengths and limitations of each platform is essential for making an informed decision, as the choice between Azure DevOps and GitHub can

have long-term implications for a development team's efficiency, security posture, and ability to scale operations.

Through an in-depth exploration of their CI/CD capabilities, automation workflows, security features, cost implications, and enterprise readiness, this paper will provide actionable guidance on when to choose one platform over the other. Whether an organization seeks a fully managed DevOps suite or a flexible, Git-based CI/CD solution, this study will help decision-makers navigate the complexities of modern software development and select the best tool for their specific needs.

2. Materials and Methods

Comparing two CI/CD platforms as comprehensive and widely used as Azure DevOps and GitHub requires a structured and systematic approach. Both platforms offer extensive capabilities, ranging from source control management and CI/CD automation to security, scalability, and cost efficiency. Their different architectures, target audiences, and integration ecosystems necessitate an analytical framework that examines their strengths and limitations across multiple dimensions. This section outlines the methodology used to compare Azure DevOps and GitHub, ensuring a balanced evaluation based on key performance indicators, usability factors, and real-world applicability.

To conduct a thorough comparison, this study adopts a multi-faceted approach that considers each platform's technical features, automation capabilities, security mechanisms, scalability, cost-effectiveness, and overall user experience. The evaluation focuses on how well these platforms support different development workflows, enterprise use cases, and DevOps adoption strategies. Since Azure DevOps and GitHub supply organizations with varying maturity levels in DevOps practices, this comparison is designed to accommodate multiple perspectives, from small development teams to large-scale enterprise environments [4,5].

One of the primary areas of comparison is CI/CD functionality, which represents the core purpose of both platforms. Azure DevOps and GitHub provide mechanisms for automating builds, running tests, and deploying applications, but they differ in their approach to workflow definition, pipeline execution, and extensibility. Azure DevOps, through Azure Pipelines, supports a highly customizable and multi-stage pipeline system with agent-based execution, offering deep integration with Microsoft technologies such as Azure Kubernetes Service (AKS), Azure Functions, and Hybrid Cloud Deployments. GitHub implements CI/CD through GitHub Actions, a YAML-based automation engine that enables developers to define workflows directly within their repositories. Each system's efficiency, flexibility, and ease of setup are carefully examined to determine which platform offers the most effective pipeline orchestration for different development scenarios.

Another crucial comparison criterion is source control and repository management. Azure DevOps and GitHub provide version control systems that allow teams to collaborate on code, track changes, and manage branching strategies. Azure DevOps supports both Git and Team Foundation Version Control (TFVC), making it particularly advantageous for organizations migrating from legacy Microsoft-based version control systems. GitHub is exclusively Git-centric, leveraging its extensive collaborative ecosystem and reputation as the leading platform for open-source development. The evaluation in this area examines aspects such as branching models, repository security, pull request mechanisms, and integration with external Git clients.

Integration capabilities play a significant role in determining the usability and extensibility of a CI/CD platform. Organizations rely on various third-party tools, cloud services, and infrastructure management solutions, making seamless integration a key factor in platform selection. Azure DevOps offers deep integration with the Microsoft ecosystem,

primarily leveraging Azure services, Active Directory authentication, and Microsoft 365 collaboration tools. It also supports connections with external cloud providers such as AWS and Google Cloud, but its native integration is strongest within the Microsoft ecosystem. With its API-driven architecture and marketplace of third-party integrations, GitHub provides a more flexible integration approach, enabling teams to connect with external CI/CD services, monitoring tools, and cloud providers in a more dynamic manner. This comparison evaluates how well each platform integrates with essential development and operations tools, including Terraform for infrastructure as code, Kubernetes for container orchestration, and security tools for vulnerability scanning and compliance enforcement [6–8].

Security and compliance form another critical dimension of this study. As software supply chain attacks and security vulnerabilities become more prevalent, CI/CD platforms must implement robust security mechanisms to ensure the integrity, confidentiality, and reliability of software releases. Azure DevOps offers built-in security features, including Azure Policy enforcement, role-based access control (RBAC), and integration with Microsoft Defender for DevOps, providing enterprise-grade security controls. GitHub Advanced Security suite includes features such as code scanning, dependency vulnerability detection, and secret scanning aimed at proactively identifying security risks within repositories. Additionally, the evaluation considers compliance with industry standards, including ISO 27001 [9], SOC 2 [10], GDPR [11], HIPAA [12], and FedRAMP [13], to determine which platform is better suited for regulated industries requiring strict security controls.

Scalability and performance are examined to assess how well each platform supports large-scale development teams, multi-repository architectures, and complex deployment scenarios. Azure DevOps, designed primarily for enterprise DevOps workflows, provides a highly scalable infrastructure that accommodates distributed teams and geographically dispersed development operations. Its ability to handle large monolithic repositories, microservices architectures, and hybrid cloud deployments is key to its appeal to enterprise customers. GitHub's decentralized and developer-friendly approach is optimized for agile teams working on smaller, fast-paced projects. It has also introduced Enterprise Cloud and Enterprise Server offerings to support larger organizations. This study evaluates pipeline execution times, artifact storage efficiency, and performance bottlenecks that could impact CI/CD reliability and speed [14,15].

Cost is another essential factor in decision-making, as organizations must balance capabilities with budget constraints. The pricing models of Azure DevOps and GitHub differ in structure, with Azure DevOps following a per-user, per-service pricing approach and GitHub adopting a tiered subscription model with free options for open-source projects. While Azure DevOps offers pay-as-you-go pricing for CI/CD execution, GitHub provides a certain number of free minutes for workflow execution with GitHub Actions, with additional costs incurred for private repositories and increased execution limits. This comparison evaluates the total cost of ownership (TCO), licensing flexibility, and potential hidden costs associated with scaling CI/CD workloads.

User experience and ease of use are also considered, particularly in interface design, developer onboarding, and workflow efficiency. With its feature-rich but complex interface, Azure DevOps has a steeper learning curve, particularly for teams new to Microsoft DevOps tools. The GitHub native interface offers a more streamlined user experience, particularly for developers already accustomed to Git workflows and open-source collaboration. This study evaluates the ease with which new users can adopt and configure CI/CD pipelines, the level of documentation and community support available, and the overall efficiency of workflow management on each platform.

By evaluating Azure DevOps and GitHub across these dimensions, this study aims to provide a comprehensive comparison that considers the technical, operational, and

financial factors organizations must consider when selecting a CI/CD platform. The goal is to identify which platform excels in specific areas and which aligns best with different organizational needs and development strategies. Through this structured comparison, software teams and IT decision-makers will gain actionable insights into how these platforms perform under various real-world conditions, ultimately helping them make an informed decision about their CI/CD adoption strategy [16–18].

The selection of CI/CD platforms was guided by their market share, unique features, and relevance to a broad range of software development teams, as visually summarized in Figure 1. The analysis focuses on Azure DevOps and GitHub, as well as the choice of a large ecosystem that includes various tools such as Jenkins, GitLab CI/CD, Atlassian Bitbucket, AWS CodePipeline, and JetBrains TeamCity. Each platform offers distinct use cases, organizational needs, and infrastructure preferences, resulting in a diverse and dynamic CI/CD landscape. According to recent Microsoft market statistics, GitHub leads with a 33% share, followed by Azure DevOps at 24%. These platforms account for over half of the market, reflecting their widespread use and significant influence. Jenkins holds a 14% share and remains a popular choice due to its open-source nature, extensibility, and robust plugin ecosystem. Jenkins' ability to integrate with any tool or platform makes it highly customizable, though this flexibility often comes at the cost of increased maintenance and operational complexity. As a result, Jenkins is particularly valuable for organizations that need technical expertise to manage and optimize their infrastructure. GitLab CI/CD, with its integrated platform, offers a compelling alternative by combining version control and project management in a single solution. Its growing adoption, reflected in a 9% market share, demonstrates its appeal to teams seeking an all-in-one platform. GitLab's emphasis on collaboration and support makes it a strong contender for modern software development teams. Atlassian's Bitbucket and JetBrains TeamCity integrate well with other tools, such as Jira and Confluence, making them a natural fit for teams already using the Atlassian suite. Its support for both Git and Mercurial version control systems, along with its built-in CI/CD capabilities, makes it an excellent choice for small to medium-sized teams. TeamCity, known for its ease of use and powerful build configurations, is another option for enterprises seeking a reliable and scalable solution. Its support for a wide range of build and deployment scenarios and user-friendly interface makes it particularly appealing to organizations with complex build requirements. Other tools, such as TravisCI and CircleCI, collectively represent a minor but significant part of the market. This distribution highlights the dominance of GitHub and Azure DevOps while underscoring the importance of alternative tools in addressing specific development and operational requirements.

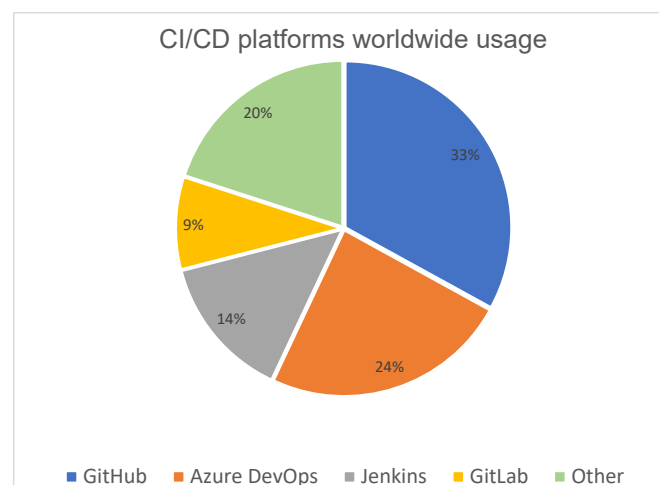


Figure 1. CI/CD platforms' worldwide usage.

3. Feature-by-Feature Comparison

Azure DevOps and GitHub, both owned by Microsoft, provide extensive DevOps capabilities. They differ significantly in their design, target audience, and approach to CI/CD, repository management, security, integration, and scalability. Azure DevOps is tailored toward enterprises seeking a structured and fully integrated DevOps solution. GitHub remains the dominant platform for open-source development and Git-based automation workflows. A direct feature comparison of these platforms reveals overlapping functionalities and key distinctions influencing how organizations choose between them.

One of the fundamental aspects of any CI/CD platform is source control and repository management. Azure DevOps supports both Git and Team Foundation Version Control (TFVC), catering to enterprises transitioning from older Microsoft-based version control systems. TFVC remains relevant for legacy systems that require centralized version control, particularly for organizations that have not fully migrated to Git. In contrast, GitHub is entirely Git-centric, emphasizing a distributed version control model that aligns with modern development practices. GitHub's repository model is inherently designed for collaboration-first workflows and an intuitive interface for pull requests, code reviews, and issue tracking, making it the preferred platform for open-source projects. Azure DevOps also supports pull requests and code reviews. The repository structure is more tightly integrated into enterprise-scale software development, particularly when managing multiple large projects under a single organization [17,19,20].

CI/CD automation is another critical feature that distinguishes these platforms. Azure DevOps provides Azure Pipelines, a robust CI/CD service that supports multi-stage pipelines, parallel job execution, YAML-based pipeline definitions, and integration with external build agents. Azure Pipelines provides deep integration with Microsoft Azure, making it an ideal choice for cloud-based deployments, Infrastructure as Code (IaC), and hybrid cloud environments. It supports containerized deployments using Docker and Kubernetes and can automate releases to Azure App Services, Azure Functions, and virtual machines. GitHub implements CI/CD through GitHub Actions, a highly flexible and modular workflow automation tool that enables developers to define custom jobs using YAML configuration files within their repositories. Unlike Azure Pipelines, which provides an end-to-end CI/CD solution with enterprise-level control, GitHub Actions is more developer-centric, offering a streamlined and event-driven approach to automation. It excels in integrating with Git workflows, making it easier for teams already accustomed to GitHub repositories to set up lightweight automation pipelines. Azure Pipelines provides more structured governance and control over build artifacts, deployment approvals, and infrastructure provisioning in large-scale enterprise deployments.

Security and compliance are increasingly critical in modern DevOps workflows, particularly for organizations operating in highly regulated environments. Azure DevOps offers enterprise-grade security features, including Azure Active Directory integration, role-based access control (RBAC), conditional access policies, and policy enforcement via Azure Policy. Security in Azure Pipelines includes artifact signing, pipeline secrets management, and access controls over deployment environments. GitHub has also significantly improved its security capabilities, particularly with the introduction of GitHub Advanced Security, which includes code scanning, dependency vulnerability detection, and secret scanning. GitHub's security model is built around GitHub Actions and Dependabot, offering automated security alerts and fixes directly within pull requests. Both platforms implement multi-factor authentication (MFA) and single sign-on (SSO). Azure DevOps provides a more enterprise-focused security model. GitHub emphasizes developer-friendly security automation that integrates natively into the repository lifecycle.

Integration capabilities further differentiate Azure DevOps and GitHub. Azure DevOps is deeply integrated with Microsoft Azure, Visual Studio, Power BI, and Microsoft Teams, making it an attractive choice for organizations using Microsoft's ecosystem. It supports connections with third-party services such as AWS, Google Cloud, and ServiceNow, but its strongest integrations remain within the Microsoft stack. GitHub is designed for flexibility and extensibility, providing thousands of third-party integrations through the GitHub Marketplace. With webhooks, REST and GraphQL APIs, and a broad ecosystem of developer tools, GitHub allows organizations to integrate with external CI/CD tools, monitoring services, and infrastructure management platforms more dynamically. The difference in integration philosophy is straightforward—Azure DevOps prioritizes seamless enterprise integration. GitHub provides a more open and customizable approach, allowing developers to integrate external tools with minimal effort.

Scalability and performance are also crucial factors in evaluating these platforms, particularly for organizations managing large codebases, globally distributed teams, and high-volume continuous integration/continuous deployment (CI/CD) workloads. Azure DevOps is designed for enterprise-scale deployments, offering on-premises (Azure DevOps Server) and cloud-hosted solutions that support large monolithic repositories, microservices architectures, and multi-region deployments. Azure Pipelines enables parallel job execution, load balancing across self-hosted agents, and fine-grained resource allocation, making it well suited for complex software development projects with stringent performance requirements. GitHub is optimized for distributed and decentralized workflows, making it ideal for agile development teams, open-source projects, and Git-based automation pipelines. GitHub Actions supports multi-runner workflows and matrix builds, but Azure DevOps provides a more structured and scalable CI/CD architecture for high-performance, enterprise-wide deployments [21].

The cost of using Azure DevOps and GitHub varies depending on the subscription model, CI/CD execution time, and required features. Azure DevOps follows a pay-per-user, per-service pricing model, allowing organizations to purchase only the services they need, including Azure Pipelines, Azure Repos, and Azure Test Plans. Azure Pipelines provides free minutes for open-source projects, but enterprise users must pay based on pipeline concurrency, agent type, and storage usage. GitHub offers a tiered pricing model, with free options for open-source repositories and GitHub Actions minutes. Private repositories and enterprise features, however, require a GitHub Enterprise subscription. GitHub is often the more cost-effective option for small teams and independent developers. Larger organizations with complex DevOps needs may find Azure DevOps to be a better long-term investment due to its enterprise-grade feature set and scalable pricing structure [22].

Another critical aspect of comparison is user experience and onboarding efficiency. With its feature-rich but complex UI, Azure DevOps presents a steeper learning curve, particularly for teams unfamiliar with Microsoft DevOps tools. Once configured, it provides a highly structured and workflow-driven approach to software development. GitHub, with its intuitive and Git-native interface, is significantly easier for developers to adopt, particularly for those already familiar with Git workflows and GitHub repositories. GitHub's modern UI, built-in collaboration tools, and seamless pull request experience make it an attractive choice for teams prioritizing simplicity and ease of use over highly structured DevOps processes.

Automated testing is a cornerstone of any CI/CD pipeline, ensuring software quality and reliability throughout the development lifecycle. Azure DevOps offers comprehensive testing tools, including Azure Test Plans, which provide end-to-end traceability for test cases, requirements, and defects. They support both manual and automated testing, with integrations for popular testing frameworks, including NUnit, xUnit, and Selenium. Azure Pipelines can execute unit tests, integration tests, and load tests as part of the CI/CD

workflow, with detailed reporting and analytics to track test results and identify bottlenecks. Additionally, Azure DevOps supports continuous testing by enabling test execution in parallel across multiple environments, reducing feedback cycles and improving deployment confidence. GitHub relies on its ecosystem of third-party integrations and GitHub Actions for automated testing. Developers can use Actions to define custom workflows for running unit tests, integration tests, and end-to-end tests using frameworks like Jest, Mocha, and Cypress. GitHub’s marketplace offers a wide range of pre-built Actions for testing, making it easy to integrate testing into CI/CD pipelines. GitHub lacks a native testing tool, such as Azure Test Plans, requiring teams to rely on external tools or custom configurations for advanced testing needs. This approach offers flexibility, but it may require additional effort to set up and maintain compared to Azure DevOps’ integrated testing solutions.

Transitioning between Azure DevOps and GitHub presents several challenges that organizations must carefully consider. One significant challenge is data migration, particularly for enterprises with large, monolithic repositories or complex CI/CD configurations. Transferring repositories, pipelines, and artifacts between platforms can be technically demanding and time-consuming, often requiring custom scripts or third-party tools to ensure data integrity and minimal downtime. Additionally, the learning curve associated with adopting a new platform can be steep. Teams accustomed to Azure DevOps’ structured, enterprise-focused workflows may find GitHub’s flexible, developer-centric approach less intuitive, and vice versa. This transition often requires extensive training and documentation to ensure a smooth onboarding process.

Another challenge lies in reconfiguring integrations. Organizations heavily invested in a single ecosystem—such as Microsoft Azure for Azure DevOps or third-party tools for GitHub—may need to overhaul their existing integrations, which can be resource-intensive. For example, migrating from Azure DevOps to GitHub may require reconfiguring connections to cloud services, monitoring tools, and security solutions. Furthermore, cost implications must be evaluated, as transitioning between platforms may involve unforeseen expenses, such as retraining staff, purchasing new licenses, or scaling infrastructure to meet performance requirements.

Workflow adjustments can pose challenges. Azure DevOps’ structured, process-driven approach may clash with GitHub’s more decentralized, collaborative model, requiring teams to adapt their workflows and communication practices. These challenges highlight the importance of thorough planning, stakeholder buy-in, and phased migration strategies to minimize disruption and ensure a successful transition.

Although Azure DevOps and GitHub share a common Microsoft heritage, their design philosophies cater to different segments of the software development ecosystem. Azure DevOps is built for enterprises that require an all-in-one DevOps solution, while GitHub is developer-centric, open-source-friendly, and optimized for Git-based automation workflows. The decision between these platforms depends on organizational priorities, infrastructure requirements, and long-term DevOps strategies. Understanding these feature-level differences, as summarized in Table 1, is critical for software teams seeking to optimize their CI/CD pipelines, security posture, and software delivery efficiency.

Table 1. Feature-by-feature comparison with key differences and considerations.

Feature Category	Azure DevOps	GitHub	Key Differences and Considerations
Version Control System	Supports Git and TFVC (Team Foundation Version Control)	Git-based only, optimized for distributed workflows	Azure DevOps is ideal for enterprises transitioning from legacy systems, GitHub is built for Git-native workflows.
Repository Management	Integrated with enterprise project structures, designed for large teams	Designed for collaborative open-source development	Azure DevOps is suited for multi-repo enterprise projects, GitHub is collaboration-first with a developer-friendly UI.

Table 1. Cont.

Feature Category	Azure DevOps	GitHub	Key Differences and Considerations
CI/CD Automation	Azure Pipelines: Multi-stage, agent-based execution, YAML-based workflows	GitHub Actions: Event-driven, GitHub-native, flexible YAML workflows	Azure Pipelines offers more structured governance, GitHub Actions provides developer-centric automation.
Security and Compliance	RBAC, Azure AD integration, artifact signing, policy enforcement	GitHub Advanced Security, Dependabot, code scanning, secret scanning	Azure DevOps is enterprise-grade with deep security integration, GitHub emphasizes proactive security automation.
Integration with Cloud and Tools	Deep integration with Azure, supports AWS and GCP but strongest in Microsoft stack	Cloud-agnostic integrates with thousands of third-party tools	Azure DevOps is best for Microsoft environments, GitHub is highly flexible and extensible.
Scalability and Performance	Optimized for enterprise deployments, hybrid cloud support, self-hosted agents	Designed for open-source and agile teams, scales with GitHub Enterprise	Azure DevOps is better for large enterprises managing multi-region deployments, GitHub is ideal for distributed teams.
Pricing Model	Pay-per-user and per-service pricing	Tiered pricing with free options for open-source projects	GitHub is more cost-effective for small teams, Azure DevOps scales better for enterprise needs.
User Experience and Onboarding	Feature-rich but complex UI requires training for new users	Intuitive, Git-native UI, easy for developers to adopt	Azure DevOps has a steeper learning curve, GitHub is more straightforward and more user-friendly.
Automated Testing	End-to-end traceability supports manual and automated testing	Relies on GitHub Actions and third-party integrations for testing	Azure DevOps provides integrated testing tools, GitHub offers flexibility through third-party integrations but requires additional setup.

4. Performance Analysis

The performance of a CI/CD platform is a critical factor in determining its suitability for modern software development workflows. Organizations rely on CI/CD pipelines to automate builds, run tests, and deploy applications across different environments. Any inefficiencies in these processes can lead to delays, increased costs, and reduced developer productivity. Evaluating the performance of Azure DevOps and GitHub requires a close examination of their efficiency, scalability, and reliability across various projects and organizational needs. These aspects are crucial in selecting a CI/CD solution that aligns with an enterprise's DevOps strategies, cloud infrastructure, and deployment models.

Efficiency is measured by how quickly and effectively a platform can execute builds, run automated tests, and deploy applications while maintaining stability and security. Azure DevOps and GitHub offer sophisticated CI/CD mechanisms. They differ in their underlying architectures and execution models, resulting in variations in pipeline execution times, caching mechanisms, and resource utilization [22,23].

Azure DevOps utilizes Azure Pipelines, which supports both Microsoft-hosted and self-hosted agents, allowing teams to tailor build environments to their specific infrastructure needs. Microsoft-hosted agents offer a preconfigured, cloud-based execution environment that supports multiple operating systems, including Windows, Linux, and macOS. Self-hosted agents allow organizations to fine-tune build performance by running pipelines on their own hardware or cloud infrastructure. This flexibility makes Azure DevOps particularly well suited for large-scale enterprise applications that require fine control over compute resources, build caching, and distributed workloads.

GitHub provides both GitHub-hosted and self-hosted runners, enabling teams to strike a balance between managed execution environments and on-premises infrastructure. GitHub Actions' execution model is optimized for smaller, incremental builds, making it more efficient for agile teams and open-source projects that prioritize rapid iteration

cycles. It offers support for parallel job execution, reusable workflows, and caching mechanisms. Its performance in handling large monolithic repositories and multi-stage enterprise builds is not as refined as Azure Pipelines, which provides more granular control over job dependencies, execution priority, and resource allocation.

Empirical data indicate that Azure Pipelines outperforms GitHub Actions for large-scale enterprise projects, particularly those requiring complex, multi-stage deployments with infrastructure-as-code, containerized workloads, and hybrid cloud integrations. However, GitHub Actions has been reported to be faster and more lightweight for small to medium-sized development teams, mainly when working within GitHub-native repositories where automation workflows are tightly integrated into the version control system. Scalability is a key differentiator between Azure DevOps and GitHub, particularly for organizations managing globally distributed teams, large monolithic repositories, and high-volume continuous integration/continuous deployment (CI/CD) workloads. As software development scales, the ability of a CI/CD platform to efficiently manage multiple concurrent builds, large datasets, and geographically distributed development teams becomes a crucial factor. Azure DevOps is designed for enterprise-scale software development, supporting multi-region deployments, hybrid cloud strategies, and complex software architectures. Organizations leveraging Azure Pipelines can configure multiple build agents across different regions, ensuring optimal load balancing and fault tolerance. Azure DevOps Server, the on-premises version of Azure DevOps, enables enterprises to host their CI/CD infrastructure within their own data centers, providing complete control over resource allocation, security policies, and performance tuning. This level of customization makes Azure DevOps particularly advantageous for large enterprises with strict regulatory requirements and high availability demands [24].

GitHub, while offering scalability through GitHub Enterprise Cloud and GitHub Enterprise Server, has historically been more developer-focused, catering primarily to distributed teams working on open-source and Git-native workflows. GitHub Enterprise Server offers on-premises hosting capabilities, but it lacks the advanced pipeline execution controls and hybrid cloud optimizations found in Azure DevOps. Additionally, large monolithic repositories with extensive histories have been reported to experience slower performance on GitHub, particularly when managing complex CI/CD pipelines with multiple dependencies and long-running builds.

Case studies from companies that have migrated from GitHub to Azure DevOps indicate that organizations dealing with multi-repository architectures, dependency-heavy software projects, and significant development teams often benefit from the structured scalability of Azure DevOps. Conversely, startups and small development teams adopting GitHub-first workflows have found that GitHub Actions scales well for their needs, particularly when combined with GitHub's cloud-based automation ecosystem.

Reliability is another critical component of performance analysis, as downtime or execution failures in CI/CD pipelines can result in significant delays in software delivery, lost productivity, and substantial financial costs. The uptime and service stability of a CI/CD platform directly impact its adoption within mission-critical applications, particularly in industries where continuous software delivery and high availability are essential. Azure DevOps benefits from Microsoft's global cloud infrastructure, ensuring high availability and fault tolerance across multiple geographic regions. Microsoft provides a 99.9% uptime SLA for Azure DevOps Services, with redundant failover mechanisms designed to minimize downtime and ensure continuous operation. Additionally, self-hosted Azure DevOps Server instances provide organizations with complete control over their availability and failover strategies, enabling the implementation of disaster recovery solutions and high-availability architectures [24–26].

As a cloud-native platform, GitHub boasts strong reliability metrics, with GitHub Enterprise Cloud offering a 99.95% uptime Service Level Agreement (SLA). However, GitHub has experienced periodic service disruptions, particularly affecting GitHub Actions execution, API availability, and repository access. Although these outages are often short-lived, they have impacted large-scale projects that rely on GitHub for mission-critical CI/CD workflows. Organizations that require strict uptime guarantees and service continuity measures may find Azure DevOps to be a more reliable option, particularly when leveraging Azure's globally distributed data centers and hybrid cloud failover capabilities. An analysis of historical downtime incidents reveals that GitHub is more susceptible to service disruptions, particularly during large-scale workflow executions during periods of peak usage. Azure DevOps, although not immune to service interruptions, benefits from Microsoft's extensive cloud resilience strategies, making it a better fit for enterprise applications where reliability is a top priority.

The performance analysis of Azure DevOps and GitHub, as shown in Table 2, reveals that both platforms excel in different areas, depending on the scale, complexity, and infrastructure needs of the development team. Azure DevOps is more efficient for large-scale enterprise projects, multi-stage deployments, and hybrid cloud environments. At the same time, GitHub Actions provides a lightweight, Git-centric automation approach that scales well for small to mid-sized teams. The scalability of Azure DevOps is superior for handling globally distributed teams and complex software architectures, whereas GitHub is optimized for fast, agile development cycles. Finally, regarding reliability, Azure DevOps offers stronger uptime guarantees and disaster recovery options, making it better suited for mission-critical applications. At the same time, GitHub remains a developer-friendly option with robust, although occasionally inconsistent, service reliability [27].

Table 2. Key differences and considerations during performance analysis.

Category	Subcategory	Azure DevOps	GitHub	Remarks
Efficiency	Build Execution Speed	Optimized for large-scale, multi-stage deployments	Faster for small to medium-sized projects	Azure DevOps excels in enterprise-scale builds; GitHub is better for agile workflows.
	Caching Mechanisms	Advanced caching for distributed workloads	Basic caching for incremental builds	Azure DevOps provides better caching for complex pipelines.
	Resource Utilization	Fine-tuned control over compute resources	Lightweight, optimized for rapid iteration	Azure DevOps offers more granular resource control.
	Pipeline Execution Times	Longer for complex, enterprise-scale projects	Shorter for small, agile workflows	GitHub is faster for smaller projects; Azure DevOps handles complexity better.
Scalability	Multi-Region Support	Yes (globally distributed infrastructure)	Limited (primarily cloud-native)	Azure DevOps supports multi-region deployments better.
	Hybrid Cloud Integration	Strong support for hybrid cloud strategies	Limited hybrid cloud optimizations	Azure DevOps is better for hybrid cloud environments.
	Large Repository Handling	Efficient for large monolithic repositories	Slower for large repositories with complex pipelines	Azure DevOps handles large repositories more efficiently.
	Concurrent Builds	Handles high-volume CI/CD workloads effectively	Struggles with high-volume, dependency-heavy projects	Azure DevOps is superior for high-volume workloads.
Reliability	Uptime SLA	99.9% (Microsoft global infrastructure)	99.95% (GitHub Enterprise Cloud)	Both offer substantial uptime, but Azure DevOps has better failover mechanisms.
	Service Disruptions	Rare, with robust failover mechanisms	Periodic disruptions during peak usage	Azure DevOps is more reliable for mission-critical applications.
	Disaster Recovery	Advanced disaster recovery options	Limited disaster recovery capabilities	Azure DevOps provides better disaster recovery solutions.
	Mission-Critical Suitability	Highly suitable for mission-critical applications	Suitable but less consistent for mission-critical workflows	Azure DevOps is preferred for mission-critical use cases.

Table 2. Cont.

Category	Subcategory	Azure DevOps	GitHub	Remarks
Use Case Fit	Enterprise-Scale Projects	Excellent for large-scale, multi-stage deployments	Less efficient for large-scale, complex projects	Azure DevOps is better for enterprise-scale needs.
	Small to Medium Teams	Overkill for small teams	Ideal for agile, small to medium-sized teams	GitHub is more suitable for smaller, agile teams.
	Open-Source Projects	Not optimized for open-source workflows	Excellent for open-source and Git-native workflows	GitHub is the preferred choice for open-source projects.
Infrastructure	Hosting Options	Microsoft-hosted and self-hosted agents	GitHub-hosted and self-hosted runners	Both offer flexibility, but Azure DevOps provides more enterprise-grade options.
	Operating System Support	Windows, Linux, macOS	Windows, Linux, macOS	Both platforms support multiple operating systems.
Advanced Features	Pipeline Isolation	Self-hosted agents in private networks, artifact signing	Ephemeral workflow environments, self-hosted runners	Azure DevOps offers better isolation for enterprise needs.
	Workflow Approval	Release approvals, immutable builds	Workflow approval processes for third-party actions	Azure DevOps provides stricter workflow controls.
	Threat Detection	Microsoft Defender for DevOps, anomaly detection	GitHub Advanced Security (code scanning, secret scanning)	Both offer strong security, but Azure DevOps integrates better with enterprise tools.

5. Security Aspects

Security has become a critical component of modern software development, particularly as cyber threats targeting CI/CD pipelines, open-source dependencies, and software supply chains have become increasingly sophisticated. Organizations adopting cloud-based DevOps solutions must ensure that their repositories, automation workflows, and deployment environments remain protected against unauthorized access, data breaches, and malicious code injections. Azure DevOps and GitHub provide robust security frameworks, but their architecture, security policies, and approach to mitigating vulnerabilities differ in ways that significantly impact an organization's security posture. Azure DevOps is designed for enterprise-scale security governance, whereas GitHub integrates security seamlessly into the developer workflow, prioritizing automation, proactive vulnerability detection, and open-source security intelligence.

A foundational aspect of security in any CI/CD platform is authentication and access control, as unauthorized access to repositories can lead to code leaks, intellectual property theft, and compromised software releases. Azure DevOps and GitHub implement multi-factor authentication (MFA), single sign-on (SSO), and role-based access control (RBAC) to ensure that users have appropriate permissions based on their roles and responsibilities [28].

Azure DevOps benefits from deep integration with Microsoft Azure Active Directory (Azure AD), enabling enterprises to enforce centralized identity management, conditional access policies, and Just-In-Time (JIT) privileged access across all DevOps activities. Organizations can configure identity governance policies that restrict repository access based on geographical locations, IP address whitelisting, and device compliance checks. This level of integration makes Azure DevOps the preferred choice for enterprises that require strict identity verification and federated authentication across multiple business units. While historically developer-centric, GitHub has significantly improved identity management and authentication security, particularly with the introduction of GitHub Enterprise Cloud and GitHub Enterprise Server. Enterprise customers can now enforce single sign-on (SSO) through SAML-based authentication providers, implement OAuth-based authorization policies, and utilize hardware security keys for added layers of authentication. GitHub and Azure DevOps also introduced fine-grained personal access tokens (PATs), which limit the scope and duration of authentication tokens, reducing the risk of token leakage or misuse.

One key difference is that GitHub repositories are, by default, publicly accessible unless explicitly marked as private, whereas Azure DevOps enforces stricter repository access controls from the outset. While GitHub Enterprise Server allows organizations to host self-contained private repositories, Azure DevOps Server provides more comprehensive access control mechanisms, including integrating on-premises IAM solutions.

Repository management and branch protection are crucial security concerns in software development, as improperly configured repositories can lead to unauthorized code changes, accidental overwrites, or malicious commits. Azure DevOps and GitHub offer branch protection rules, mandatory code reviews, and commit signing, but their implementation differs in complexity and enforcement capabilities [29].

Azure DevOps offers branch security policies that prevent unauthorized direct commits to protected branches, ensuring that every change undergoes review, validation, and approval before being integrated. Organizations can enforce gated check-ins, requiring developers to run automated build tests before merging code. Additionally, Azure DevOps enables branch locking, explicit permission settings per branch, and hierarchical repository security policies, providing enterprises with greater control over who can modify critical parts of the codebase. GitHub implements protected branches and requires reviewers to approve pull requests before they can be merged into the main branch. It also includes draft pull requests, which allow developers to share code changes in progress without allowing them to be merged. A significant security advantage of GitHub is its integration with GitHub Actions for security enforcement, where teams can automate security checks, code linting, and compliance scans directly within pull requests.

One unique feature in GitHub is committed signature verification, which enables organizations to require signed commits using GPG encryption keys or SSH keys, preventing spoofed commits and unauthorized changes. While supporting commit verification, Azure DevOps emphasizes enterprise-grade repository controls, including project-level access hierarchies and integration with Azure Policy for compliance checks.

Both platforms offer automated security scanning features that proactively detect vulnerabilities in source code, dependencies, and CI/CD configurations. Azure DevOps integrates with Microsoft Defender for DevOps, providing real-time threat detection, anomaly detection in pipelines, and integration with Microsoft Security Graph to identify and mitigate security risks across the software development lifecycle. This makes Azure DevOps particularly effective for organizations that require advanced threat intelligence, automated security alerts, and compliance-driven security assessments.

GitHub Advanced Security suite provides code scanning, Dependabot, and secret scanning, enabling automated vulnerability detection at the repository level. Dependabot automatically monitors dependencies for security vulnerabilities, generating pull requests with patched versions of affected libraries. GitHub's secret scanning feature detects leaked API keys, database credentials, and authentication tokens within repositories, helping developers prevent accidental exposure of sensitive information [29,30].

CI/CD pipelines are often targeted by attackers attempting to inject malicious code, compromise build artifacts, or alter deployment configurations. Azure DevOps and GitHub implement different strategies for securing CI/CD workflows, each with unique advantages and potential limitations.

Azure DevOps provides pipeline isolation mechanisms, allowing teams to configure self-hosted agents within private networks and restrict access to external dependencies and unauthorized build environments. Enterprises can enforce artifact signing, immutable build verification, and release approvals, ensuring that only validated artifacts are deployed to production.

GitHub Actions, which offers flexibility and automation, has faced security concerns related to untrusted third-party actions, where malicious workflows can be introduced into repositories. To mitigate this, GitHub introduced workflow approval processes, ephemeral workflow environments, and enhanced access restrictions for self-hosted runners. However, Azure DevOps remains the more substantial option for enterprises requiring fine-grained CI/CD security governance.

As software supply chain attacks become increasingly prevalent, both platforms have implemented security features to mitigate risks associated with open-source dependencies, third-party integrations, and deployment workflows. Azure DevOps integrates with Microsoft's Zero Trust security model, ensuring end-to-end security validation of source code, build artifacts, and infrastructure configurations before deployment. Microsoft's Software Bill of Materials (SBOM) compliance framework provides organizations with complete visibility into dependencies, version tracking, and security audits.

GitHub has also taken a strong stance on supply chain security, particularly with the GitHub Advisory Database, npm security advisories, and GitHub Actions for supply chain verification. GitHub's approach prioritizes automation, open-source security intelligence, and real-time vulnerability patching, while Azure DevOps focuses on compliance, governance, and risk management frameworks tailored for large enterprises [31,32].

Azure DevOps and GitHub provide strong security capabilities, but their primary focus areas differ significantly, as reflected in Table 3. Azure DevOps is designed for enterprises that require strict compliance, centralized security governance, and Microsoft-native threat intelligence, making it the preferred option for regulated industries and complex DevOps environments. GitHub's developer-first approach excels in security automation, proactive vulnerability detection, and seamless integration with Git-native workflows, making it ideal for agile teams, open-source projects, and rapid development cycles. Organizations must carefully assess their security priorities, regulatory requirements, and DevSecOps strategies to determine which platform aligns best with their software security objectives.

Table 3. Security aspects are divided by category.

Category	Subcategory	Azure DevOps	GitHub	Remarks
Authentication and Access Control	Multi-Factor Authentication (MFA)	Supported	Supported	Both platforms enforce MFA for enhanced security.
	Single Sign-On (SSO)	Integrated with Azure AD	SAML-based SSO (Enterprise Cloud/Server)	Azure DevOps has deeper integration with Azure AD for enterprise needs.
	Role-Based Access Control (RBAC)	Advanced RBAC with hierarchical permissions	Basic RBAC with repository-level permissions	Azure DevOps offers more granular access control.
	Fine-Grained Access Tokens (PATs)	Supported (limited scope and duration)	Supported (limited scope and duration)	Both platforms reduce token leakage risks.
Repository Security	Default Repository Visibility	Private by default	Public by default (unless explicitly set to private)	Azure DevOps enforces stricter access controls from the outset.
	Branch Protection	Prevents unauthorized direct commits, gated check-ins, and branch locking	Protected branches, required reviewers, draft pull requests	Azure DevOps provides more enterprise-grade branch controls.
	Commit Signing	Supported	GPG/SSH commit signature verification	GitHub enforces signed commits for better authenticity.
	Code Review Enforcement	Mandatory code reviews, hierarchical repository policies	Required reviewers, automated security checks in pull requests	Both platforms enforce code reviews, but GitHub integrates automation better.
	Repository Access Controls	IP whitelisting, geographical restrictions, device compliance checks	Self-hosted private repositories, SAML-based access controls	Azure DevOps offers more comprehensive access controls.

Table 3. Cont.

Category	Subcategory	Azure DevOps	GitHub	Remarks
CI/CD Pipeline Security	Pipeline Isolation	Self-hosted agents in private networks, artifact signing	Ephemeral workflow environments, self-hosted runners	Azure DevOps offers enhanced isolation for enterprise needs.
	Workflow Approval	Release approvals, immutable builds	Workflow approval processes for third-party actions	Azure DevOps enforces stricter workflow controls.
	Threat Detection	Microsoft Defender for DevOps, anomaly detection	GitHub Advanced Security (code scanning, Secret Scanning)	Both offer strong threat detection, but Azure DevOps integrates better with enterprise tools.
	Malicious Code Prevention	Immutable build verification, gated check-ins	Automated security checks in pull requests	Azure DevOps is more decisive for preventing malicious code injections.
Supply Chain Security	Dependency Monitoring	Integration with Microsoft Security Graph, SBOM compliance	Dependabot for dependency updates, npm security advisories	GitHub excels in automated dependency patching.
	Vulnerability Detection	Real-time threat detection, compliance-driven security assessments	Code scanning, Secret Scanning, GitHub Advisory Database	Both platforms provide robust vulnerability detection.
	Third-Party Integrations	Zero Trust model, end-to-end security validation	GitHub Actions for supply chain verification	Azure DevOps is better for compliance-driven enterprises.
	Open-Source Security	Limited focus on open-source workflows	Strong focus on open-source security intelligence	GitHub is better suited for open-source projects.
Compliance and Governance	Regulatory Compliance	Tailored for regulated industries (HIPAA [12], GDPR [11])	Limited compliance frameworks	Azure DevOps is preferred for compliance-heavy industries.
	Audit Logging	Comprehensive audit logs, integration with Azure Policy	Basic audit logging	Azure DevOps provides better audit capabilities.
	Disaster Recovery	Advanced disaster recovery options, high-availability architectures	Limited disaster recovery capabilities	Azure DevOps is stronger for mission-critical applications.

6. Pricing and Cost Considerations

Cost is a significant factor in choosing a CI/CD platform, as organizations must balance functionality, scalability, and security with the financial impact of adopting and maintaining a DevOps solution. The pricing models of Azure DevOps and GitHub differ significantly, reflecting their distinct approaches to software development workflows, as summarized in Table 4. While Azure DevOps follows a pay-per-user and per-service model, offering flexibility in selecting individual components, GitHub adopts a tiered pricing structure that differentiates between open-source and private repositories, team collaboration features, and enterprise-grade security offerings. A detailed cost analysis is necessary to understand the total cost of ownership, hidden expenses, and cost-effectiveness of each platform across different organizational sizes and use cases.

Azure DevOps and GitHub offer distinct pricing models for users, ranging from individual developers to large-scale enterprises. Azure DevOps is structured around modular services, allowing organizations to pay only for the features they need. GitHub's pricing tiers bundle features together, requiring organizations to upgrade plans to access specific functionalities. Azure DevOps offers a free tier, which includes basic features for up to five users, providing limited access to Azure Repos, Azure Pipelines, and Azure Boards. Additionally, organizations must purchase separate Azure DevOps Services, each billed individually. Azure Repos, which provides version control, is free for unlimited public and private repositories. Azure Pipelines, which powers CI/CD, includes 1800 free pipeline minutes per month for public repositories, but private repositories require payment for

additional execution time. Azure Boards, which serves as a project management tool, is included in all paid plans. Azure Test Plans, which support automated testing, are billed separately based on the number of users. Azure Artifacts, which provides package management, charges organizations based on storage and usage [32,33].

Table 4. Pricing and cost model comparison.

Category	Subcategory	Azure DevOps	GitHub	Remarks
Pricing Models	Free Tier	Free for up to 5 users (basic features)	Free for unlimited public/private repositories (limited CI/CD minutes)	GitHub's free tier is more generous for small teams and open-source projects.
	Pay-Per-User	Yes (beyond five users)	Yes (Team and Enterprise plans)	Azure DevOps charges per user; GitHub bundles features in tiered plans.
	Modular Pricing	Yes (pay for individual services like Pipelines, Test Plans, and Artifacts)	No (bundled features in tiers)	Azure DevOps offers more flexibility for enterprises.
	Enterprise Pricing	Custom pricing for advanced features (Azure Test Plans, Defender for DevOps)	GitHub Enterprise Cloud/Server (custom pricing for advanced security)	Both require enterprise plans for advanced features.
CI/CD Pipeline Costs	Free CI/CD Minutes	1800 free minutes/month (public repos)	2000 free minutes/month (Free plan)	GitHub offers slightly more free minutes for small teams.
	Private Repo Costs	Pay for additional pipeline minutes	Pay for additional minutes (higher costs for macOS runners)	Azure DevOps is cheaper for private repos with self-hosted agents.
	Self-Hosted Runners	Supported (reduces costs for private infrastructure)	Supported (reduces costs but has higher per-minute pricing for cloud runners)	Azure DevOps is more cost-effective for self-hosted pipelines.
	Concurrency Costs	Pay for additional concurrent jobs	Pay for additional concurrent jobs (Enterprise plan required for priority)	Azure DevOps offers more flexibility for scaling concurrency.
Enterprise Features	Advanced Security	Integrated with Microsoft Defender for DevOps (additional cost)	GitHub Advanced Security (Enterprise plan required)	Both require enterprise plans for advanced security.
	Compliance Support	Built-in compliance (ISO 27001 [9], SOC 2 [10], GDPR [11], HIPAA [12], FedRAMP [13])	Substantial compliance (Enterprise plan required)	Azure DevOps has better built-in compliance support.
	Hybrid Cloud Support	Strong support for hybrid cloud strategies	Limited hybrid cloud optimizations	Azure DevOps is better for hybrid cloud environments.
Hidden Costs	Storage Costs	Charges for additional storage (repos, artifacts, pipeline logs)	Usage limits for API calls, Actions execution, dependency scans	Both platforms have hidden costs for storage and usage.
	Network Bandwidth	Charges for data transfer beyond free limits	Usage limits for API calls and Actions execution	Azure DevOps may incur higher network costs for large-scale projects.
	Training and Onboarding	Higher training costs due to complex interface	Lower training costs (developer-friendly workflows)	GitHub reduces onboarding costs for small teams.
Cost-Effectiveness	Small Teams/Open Source	Limited free tier (5 users)	Free tier with unlimited public/private repos	GitHub is more cost-effective for small teams and open-source projects.
	Large Enterprises	Flexible pricing, self-hosted agents, hybrid cloud support	Higher costs for enterprise features and cloud-hosted runners	Azure DevOps is more cost-effective for large enterprises.
	Compliance-Heavy Industries	Better built-in compliance, integration with Microsoft security tools	Requires Enterprise plan for advanced compliance	Azure DevOps is preferred for regulated industries.
	Developer Productivity	Higher training costs, but better for complex workflows	Lower training costs, ideal for agile workflows	GitHub is better for developer productivity in small teams.

Azure DevOps follows a per-user pricing model, where additional users beyond the free five-user limit require paid licenses. Organizations can also purchase self-hosted pipeline agents, allowing them to run CI/CD jobs on their infrastructure at a reduced cost compared to Microsoft-hosted agents. GitHub's pricing structure is tier-based, with three primary plans: Free, Team, and Enterprise. The Free plan supports unlimited public and private repositories but limits CI/CD execution time. The Team plan introduces collaborative features, security enforcement tools, and better repository management at a fixed per-user price. The Enterprise plan is designed for large-scale organizations, offering advanced security features, compliance management, and dedicated support.

Unlike Azure DevOps' modular pricing, GitHub's Enterprise Cloud is charged per user monthly, bundling security tools, CI/CD execution, and advanced collaboration features together. While this simplifies pricing, it also means that organizations may incur costs for features they do not actively use, potentially leading to inefficiencies for specific DevOps teams.

One of the most critical pricing considerations in any DevOps platform is the cost of running CI/CD pipelines, as build and deployment operations consume compute resources that are billed based on execution time, concurrency, and the type of runner used. Azure DevOps offers 1800 free pipeline minutes per month for public repositories, while private repositories incur a charge for additional execution time. Organizations needing concurrent jobs or self-hosted agents must purchase additional pipeline execution capacity. The cost structure is influenced by the type of virtual machines used, with Linux-based builds being cheaper than Windows or macOS builds.

GitHub Actions offers a different pricing model, granting free CI/CD minutes but applying limits based on the subscription tier. GitHub Free users receive 2000 CI/CD minutes monthly, while GitHub Pro and Enterprise users receive higher quotas and priority execution access. Organizations that use self-hosted runners can reduce costs by executing CI/CD workflows on their own hardware or cloud infrastructure rather than relying on GitHub's hosted runners. However, GitHub Actions charges differently based on the runner's operating system, with macOS runners being significantly more expensive than Linux-based or Windows-based runners [30,31,34].

A key cost consideration is that Azure DevOps provides more flexible pricing for self-hosted pipelines, allowing organizations to reduce CI/CD execution costs if they have their own cloud or on-premises infrastructure. GitHub Actions, while allowing self-hosted runners, is still more expensive for enterprises that require extensive pipeline execution at scale due to GitHub's higher cost-per-minute pricing model for cloud-hosted runners.

For enterprises, security, compliance, and scalability features are crucial when evaluating the financial impact of adopting a DevOps platform. GitHub Advanced Security, which includes code scanning, secret scanning, and Dependabot security alerts, is only available under GitHub Enterprise Cloud, requiring organizations to purchase the highest-tier plan to access these features. Similarly, GitHub Enterprise Server, an on-premises version of GitHub, requires custom pricing and additional infrastructure costs, making it less cost-effective for organizations that need private repository hosting with advanced security capabilities.

Azure DevOps provides enterprise-grade security and compliance by integrating with Microsoft security services. Organizations requiring advanced governance policies, custom compliance configurations, and premium security analytics may need additional Azure services, such as Microsoft Defender for DevOps, Azure Policy, and Azure Sentinel, all of which incur additional costs.

The cost of meeting compliance standards must also be considered for organizations operating in highly regulated industries such as finance, healthcare, and government.

Azure DevOps benefits from Microsoft's extensive compliance framework, supporting certifications such as ISO 27001 [9], SOC 2 [10], GDPR [11], HIPAA [12], and FedRAMP [13]. GitHub also maintains strong compliance standards, but enterprise security and governance features are only available under GitHub Enterprise Cloud or GitHub Enterprise Server. While Azure DevOps has better built-in compliance support, it requires integration with other Microsoft security tools. GitHub requires an enterprise subscription to access advanced security features, potentially increasing costs depending on the organization's security and compliance needs [35].

Beyond the base subscription fees, organizations must also consider hidden costs, including data storage, network bandwidth, and extended support plans. Azure DevOps charges for additional storage beyond the free limit for repositories, artifacts, and pipeline logs. GitHub applies usage limits for API calls, GitHub Actions execution, and dependency security scans.

An important long-term cost consideration is developer onboarding and productivity. GitHub's intuitive interface and developer-friendly workflows reduce training costs and onboarding time. Azure DevOps, due to its more complex interface and enterprise feature set, may require more training and configuration effort. For large organizations, the time required to train teams on a CI/CD platform translates into indirect costs, making GitHub a more cost-effective solution for small teams and Azure DevOps a better long-term investment for enterprises that need scalability and compliance integration.

The cost-effectiveness of Azure DevOps and GitHub depends on the organization's size, the complexity of its development workflows, and its infrastructure strategy. Azure DevOps provides a more flexible pricing model, allowing organizations to select individual services and optimize costs using self-hosted agents. It is ideal for enterprises that require full DevOps integration within the Microsoft ecosystem. GitHub's tiered pricing model offers a more streamlined and predictable cost structure. Enterprise features and advanced security tools are locked behind higher-tier plans, making them potentially more expensive for large organizations that require premium security and governance controls.

For individual developers, open-source projects, and small teams, GitHub is often the more cost-effective choice due to its free public repository hosting and built-in CI/CD minutes. However, Azure DevOps provides better long-term cost efficiency for large enterprises that need fine-grained DevOps control, hybrid cloud flexibility, and integration with Microsoft security solutions, particularly for organizations managing large-scale software projects and complex compliance requirements.

7. Use Cases and Industry Adoption

Choosing CI/CD platforms depends on several factors, including project scale, team structure, security requirements, cloud infrastructure, and industry regulations. Azure DevOps and GitHub offer robust DevOps solutions. Still, their adoption varies significantly based on the type of organizations using them, the complexity of their software development pipelines, and the level of integration required with existing ecosystems. Understanding how these platforms are utilized across various industries helps identify the tool that best suits specific development environments and operational needs.

Enterprises with complex software architectures, multi-cloud environments, and strict compliance requirements often prioritize Azure DevOps due to its deep integration with the Microsoft ecosystem, enterprise-grade security features, and structured DevOps capabilities. Large corporations operating in finance, healthcare, government, and manufacturing require a fully managed DevOps solution that supports end-to-end software development lifecycle management, compliance enforcement, and hybrid cloud deployments. Azure DevOps is designed to meet these needs, offering Azure Pipelines for scalable CI/CD

automation, Azure Boards for project tracking, Azure Repos for secure code storage, and Azure Test Plans for quality assurance [34–36].

Many Fortune 500 companies and large government agencies leverage Azure DevOps to manage distributed teams and mission-critical applications. Enterprises that rely on on-premises infrastructure or hybrid cloud models often choose Azure DevOps Server, which allows them to maintain complete control over their DevOps environments while integrating seamlessly with cloud-based resources. Regulated industries like banking, healthcare, and defense benefit from Azure DevOps' compliance certifications, identity and access management (IAM) controls, and advanced security configurations.

Azure DevOps offers enhanced support for transitioning from traditional software development models to modern CI/CD practices, particularly for companies operating large-scale, monolithic applications with legacy dependencies. Organizations requiring complex deployment workflows, multi-stage approvals, and governance policies favor Azure Pipelines, which allows customized deployment strategies with fine-grained access control. Software vendors and system integrators that work with Microsoft Azure services often prefer Azure DevOps because it offers first-class integration with Azure Kubernetes Service (AKS), Azure Functions, and other cloud-native services.

GitHub has long been the preferred platform for open-source development, offering a developer-friendly experience, seamless Git workflows, and built-in collaboration tools. The platform hosts millions of public repositories, enabling global developer communities, research institutions, and independent software projects to contribute to open-source initiatives. Organizations prioritize community-driven development, transparency, and public code sharing, relying on GitHub's repository model and collaboration features [37,38].

Startups and small development teams also favor GitHub, as it provides free repository hosting, built-in CI/CD automation through GitHub Actions, and an intuitive interface for managing code repositories. Compared to Azure DevOps, GitHub has a lower entry barrier, making it more accessible for early-stage software projects, experimental development, and agile workflows. Tech startups developing cloud-native applications, AI-driven services, and mobile apps often find GitHub's flexibility and marketplace integrations helpful in accelerating product development.

Companies that follow a GitOps approach, where infrastructure and application deployments are managed using Git as the only source of truth, also prefer GitHub for version-controlled infrastructure automation. Cloud-native software teams using Kubernetes, Terraform, or serverless architectures often leverage GitHub Actions to automate deployment pipelines while integrating with third-party tools such as HashiCorp Vault for secrets management and Prometheus for monitoring.

Organizations adopting a multi-cloud or hybrid cloud strategy often need flexibility in managing CI/CD pipelines across different cloud environments. While Azure DevOps is deeply integrated with Microsoft Azure, it also supports AWS and Google Cloud, allowing enterprises to build hybrid DevOps architectures that span multiple cloud providers. Companies that deploy applications across Azure, AWS, and on-premises data centers often prefer Azure DevOps for its enterprise-wide DevOps governance. GitHub's cloud-agnostic approach makes it a strong choice for organizations that use multi-cloud deployments but want a Git-native workflow for version control and automation. Companies that rely on Google Cloud Platform (GCP) or AWS for hosting and deployment often integrate GitHub Actions with Terraform, AWS Lambda, and Google Kubernetes Engine (GKE) to manage multi-cloud infrastructure as code [39].

Highly regulated industries, such as financial services, healthcare, pharmaceuticals, and government agencies, require stringent security controls, comprehensive compliance auditing, and robust governance policies for DevOps operations. Azure DevOps is the

preferred choice in these industries due to its comprehensive security model, integration with Azure Security Center, and support for various compliance frameworks, including ISO 27001 [9], SOC 2 [10], HIPAA [12], and FedRAMP [13]. Organizations that must adhere to government regulations and corporate security policies utilize Azure DevOps to enforce compliance across CI/CD pipelines, infrastructure-as-code, and artifact management.

GitHub developer-centric security features, such as GitHub Advanced Security, Dependabot, and secret scanning, are ideal for organizations that want proactive security measures integrated into their development workflow. Security-conscious software teams that rely on automated vulnerability scanning, open-source security advisories, and real-time security patches benefit from GitHub's automated security scanning capabilities. However, for companies that require stronger access controls, compliance management, and enterprise-wide DevSecOps strategies, Azure DevOps provides more structured governance and integration with Microsoft security tools.

Specific industries, such as game development, embedded systems engineering, and industrial automation, require specialized software development environments that integrate with custom-built tools, graphics engines, and hardware-specific toolchains. Many AAA game studios and software vendors prefer Azure DevOps due to its compatibility with Microsoft development tools, including Visual Studio, DirectX, and the HoloLens SDK. Game developers building for Xbox, Windows, and cloud-based gaming services benefit from Azure DevOps' structured pipeline automation and integration with Microsoft's developer ecosystem [40,41].

Conversely, independent game developers and open-source game engines frequently utilize GitHub to host their source code and collaborate with community contributors. GitHub is widely used in indie game development, academic research projects, and experimental software engineering due to its Git-based repository management and accessibility for developers working on cross-platform projects.

The industry adoption of Azure DevOps and GitHub can be broadly categorized based on whether an organization prioritizes enterprise DevOps governance or developer-driven collaboration. Enterprises that require structured DevOps environments, compliance enforcement, and large-scale software delivery pipelines choose Azure DevOps due to its robust security framework, integration with corporate IT policies, and multi-stage CI/CD pipeline capabilities. Organizations focusing on developer agility, Git-based automation, and community collaboration prefer GitHub, particularly for projects emphasizing open-source development, cloud-native workflows, and iterative software innovation.

The decision between Azure DevOps and GitHub depends on organizational goals, project scale, security requirements, and cloud strategy, as reflected in Table 5. Azure DevOps is best suited for enterprises, government agencies, and regulated industries that require a comprehensive DevOps suite with structured workflows, automated compliance, and support for hybrid cloud environments. GitHub is preferred for open-source development, agile software teams, and organizations prioritizing Git-native automation and cloud-agnostic CI/CD pipelines. Companies must carefully evaluate their development methodologies, infrastructure preferences, and long-term DevOps strategies to determine which platform aligns best with their operational needs and software delivery objectives [42,43].

Table 5. Use cases and technical adoption decision scenarios.

Category	Subcategory	Azure DevOps	GitHub	Remarks
Enterprise Use Cases	Large-Scale Applications	Ideal for monolithic applications, legacy systems, and complex workflows	Less suited for large-scale, dependency-heavy projects	Azure DevOps excels in delivering enterprise-scale software.
	Multi-Stage Deployments	Supports complex deployment workflows with fine-grained access control	Limited support for multi-stage approvals and governance	Azure DevOps is better suited for enterprises with stringent governance requirements.
	Hybrid Cloud Support	Strong support for hybrid cloud strategies and on-premises infrastructure	Limited hybrid cloud optimizations	Azure DevOps is preferred for hybrid cloud environments.
	Fortune 500 Adoption	Widely adopted by Fortune 500 companies and government agencies	Less common in large enterprises with strict compliance needs	Azure DevOps is the go-to for large, regulated organizations.
Open-Source and Small Teams	Open-Source Development	Limited focus on open-source workflows	Preferred platform for open-source projects and public repositories	GitHub dominates the open-source ecosystem.
	Startups and Small Teams	Overkill for small teams due to complexity	Ideal for startups, small teams, and agile workflows	GitHub is more accessible and cost-effective for small teams.
	Developer Experience	Complex interface, higher learning curve	Intuitive interface, developer-friendly workflows	GitHub reduces onboarding time and training costs.
Multi-Cloud and Hybrid Cloud	Multi-Cloud Support	Supports AWS, Google Cloud, and Azure with hybrid DevOps architectures	Cloud-agnostic integrates with AWS, GCP, and Terraform	Both support multi-cloud environments, but Azure DevOps is better suited for hybrid setups.
	GitOps Approach	Limited native support for GitOps	Strong support for GitOps with GitHub Actions and Terraform integrations	GitHub is better for GitOps-driven workflows.
Regulated Industries	Compliance and Security	Strong compliance support (ISO 27001 [9], SOC 2 [10], HIPAA [12], FedRAMP [13])	Requires Enterprise plan for advanced compliance features	Azure DevOps is better for regulated industries.
	Government and Defense	Widely adopted by government agencies and defense contractors	Less common in highly regulated industries	Azure DevOps meets strict government security requirements.
	Healthcare and Finance	Preferred for healthcare, finance, and pharmaceuticals	Limited adoption in highly regulated sectors	Azure DevOps integrates better with compliance frameworks.
Specialized Industries	Game Development	Strong integration with Microsoft tools (Visual Studio, DirectX, HoloLens SDK)	Popular among indie game developers and open-source game engines	Azure DevOps is better for AAA game studios, and GitHub is for indie developers.
	Embedded Systems	Supports custom-build tools and hardware-specific toolchains	Limited support for specialized hardware environments	Azure DevOps is better for industrial automation and embedded systems.
	AI and Cloud-Native Development	Strong integration with Azure AI and cloud-native services	Ideal for AI-driven services, mobile apps, and cloud-native workflows	GitHub is better for agile, cloud-native development.
Industry Adoption Trends	Enterprise Adoption	Dominates in large enterprises, regulated industries, and government agencies	Limited adoption in enterprises with strict compliance needs	Azure DevOps is the choice for enterprise-grade DevOps.
	Developer Community	Limited community-driven development	Thriving open-source community and public repositories	GitHub is the hub for open-source collaboration.
	Agile and Iterative Development	Less suited for rapid iteration and experimentation	Ideal for agile teams, iterative development, and experimental projects	GitHub is better for fast-paced, agile workflows.

8. Future Directions

As the landscape of CI/CD platforms continues to evolve, Azure DevOps and GitHub are expected to introduce new capabilities, optimizations, and integrations to address emerging software development challenges. Organizations adopting DevOps methodolo-

gies are constantly looking for improvements in automation, security, performance, and cloud-native development, making it crucial for these platforms to adapt to industry trends. The future of CI/CD tooling will be shaped by advancements in artificial intelligence (AI)-driven automation, enhanced security practices, hybrid and multi-cloud strategies, and deeper integrations with DevSecOps workflows.

One of the most significant trends shaping the future of CI/CD is the integration of AI and machine learning into DevOps workflows. Microsoft has already begun incorporating AI-powered automation features into GitHub Copilot, which assists developers in writing code by providing AI-based suggestions. Future enhancements could extend AI-driven capabilities into CI/CD pipelines, where machine learning models optimize test execution, deployment strategies, and pipeline performance tuning. Azure DevOps is expected to introduce AI-powered tools for automated anomaly detection, self-healing CI/CD pipelines, and predictive analytics for build failures. GitHub, with its strong developer-focused ecosystem, may further leverage AI-driven automation in GitHub Actions to suggest optimized workflows, detect inefficiencies in CI/CD pipelines, and even automate compliance verification through intelligent code scanning.

Security remains a top priority for modern DevOps, and both platforms will expand their security and compliance capabilities to combat evolving threats. GitHub Advanced Security has already made strides in code scanning, secret detection, and dependency vulnerability tracking. Future developments could see GitHub integrating with Zero Trust security models, real-time threat intelligence feeds, and automated security policy enforcement across repositories. On the other hand, Azure DevOps is expected to enhance enterprise security controls, mainly through deeper integration with Microsoft Defender for DevOps, improved privileged access management, and further alignment with government and industry compliance standards. As software supply chain attacks become increasingly sophisticated, both platforms must implement stronger integrity verification mechanisms, such as signed builds, automated provenance tracking, and blockchain-backed audit trails, to ensure artifact security.

The rise of hybrid and multi-cloud DevOps strategies will also shape the future evolution of both platforms. Being deeply integrated with Microsoft Azure, Azure DevOps may introduce more seamless multi-cloud deployment capabilities, allowing organizations to manage CI/CD pipelines across AWS, Google Cloud, and private cloud environments more effectively. GitHub is likely to expand its support for Kubernetes-native deployments, GitOps workflows, and infrastructure-as-code automation, making it a preferred choice for organizations seeking flexibility across multiple cloud providers. Future updates could bring enhanced Terraform integration, automated multi-cloud configuration validation, and intelligent cost optimization recommendations for cloud deployments.

In addition to cloud integration, the role of DevSecOps in CI/CD workflows will continue to expand, resulting in tighter security and governance integrations across both platforms. GitHub Actions may introduce native security scanning workflows that automatically remediate vulnerabilities, while Azure DevOps could implement enterprise-wide security baselines that enforce secure development best practices across organizations. The expansion of compliance automation, where CI/CD pipelines automatically validate builds against regulatory requirements, is also expected to become a core feature of both platforms.

Developer experience and workflow efficiency will remain a crucial area of innovation. GitHub is likely to enhance its UI/UX, making repository and workflow management even more intuitive. Azure DevOps may focus on improving cross-team collaboration through enhanced integration with Microsoft Teams, Azure Boards, and AI-powered project management insights. The emergence of low-code and no-code development trends may

also push both platforms to introduce simplified pipeline configuration tools, allowing non-technical users to set up basic automation workflows with minimal scripting.

DevOps continues to mature, and the evolution of Azure DevOps and GitHub will depend on how well they adapt to AI-driven automation, security advancements, hybrid cloud trends, and DevSecOps best practices. Organizations must stay informed about new developments in both platforms to ensure their CI/CD strategies remain competitive, secure, and scalable in an increasingly complex software development ecosystem.

9. Results

The choice between Azure DevOps and GitHub is a strategic decision that depends on an organization's size, development workflows, security requirements, compliance obligations, and infrastructure preferences. Both platforms offer robust CI/CD automation, version control, and security mechanisms, yet they differ significantly in their target audience, enterprise readiness, and integration capabilities. Throughout this analysis, key findings have demonstrated the strengths and limitations of each platform, offering a clear perspective on where they excel and where they might be less suitable based on industry-specific needs.

Azure DevOps is the preferred solution for enterprises that require a structured, full-featured DevOps environment with comprehensive project management tools, compliance-driven security policies, and deep integration with Microsoft services. Large corporations, government agencies, and organizations in highly regulated industries such as finance, healthcare, and defense benefit from its enterprise-scale governance, identity and access management (IAM) policies, and hybrid cloud compatibility. Its modular structure allows companies to select individual services, such as Azure Repos for version control, Azure Pipelines for CI/CD automation, and Azure Test Plans for software quality assurance, making it a flexible yet highly controlled environment for complex software development lifecycles. Additionally, Azure DevOps is well suited for multi-cloud and hybrid cloud strategies, offering seamless connectivity with Microsoft Azure and external cloud providers, including AWS and Google Cloud. Organizations that require on-premises DevOps capabilities also benefit from the Azure DevOps Server, allowing them to maintain complete control over their software development infrastructure until complying with strict data sovereignty regulations.

GitHub is the dominant platform for open-source development, agile software teams, and Git-native automation workflows. Its developer-friendly interface, strong collaborative ecosystem, and integration with GitHub Actions for CI/CD automation make it an ideal choice for startups, independent developers, and technology companies prioritizing rapid iteration and cloud-based development. The GitHub Marketplace and API ecosystem provide extensive customization options, allowing teams to integrate third-party DevOps tools, infrastructure automation services, and cloud-native deployment strategies. GitHub's security model, particularly with GitHub Advanced Security, Dependabot, and secret scanning, is optimized for proactive vulnerability management and automated security enforcement within Git workflows. Its cloud-first approach makes it an excellent choice for companies that rely on serverless architectures, Kubernetes-based deployments, and cloud-native applications.

A key distinction between the two platforms lies in their security and compliance frameworks. Azure DevOps is designed for organizations that require strict security controls, enterprise-wide compliance enforcement, and advanced identity management policies. Its integration with Microsoft Defender for DevOps, Azure Policy, and Zero Trust security models ensures that software supply chain risks, CI/CD pipeline vulnerabilities, and access control violations are mitigated through centralized governance mechanisms.

GitHub, offering robust security automation tools, remains more developer-driven, with security scanning and dependency monitoring primarily focused on repository-level protection rather than enterprise-wide governance. Azure DevOps provides a more structured security framework for organizations operating in highly regulated industries that require full audit trails, custom compliance policies, and secure artifact storage.

Cost is another significant factor in selecting a CI/CD platform, and the pricing models of Azure DevOps and GitHub reflect their different approaches to software development. Azure DevOps follows a pay-per-user and per-service model, allowing organizations to pay only for the features they use. This makes it cost-effective for enterprises that need scalable DevOps solutions tailored to specific business units. However, it can become more expensive for organizations that require extensive CI/CD execution and testing infrastructure, particularly for teams running thousands of concurrent pipeline jobs. GitHub's tiered subscription model is often more budget-friendly for small teams and open-source projects, offering free public repositories, built-in CI/CD minutes, and collaborative tools at no additional cost. However, enterprise features such as GitHub Advanced Security and enterprise-wide policy enforcement require higher-tier subscriptions, which can increase costs for large organizations that require premium security and governance features.

The industry adoption of Azure DevOps and GitHub further reinforces their respective strengths. Enterprises that require end-to-end software lifecycle management, strict governance policies, and integration with legacy Microsoft development tools tend to prefer Azure DevOps for its enterprise-oriented capabilities. Cloud-native companies, startups, and open-source communities are drawn to GitHub due to its Git-centric workflows, community collaboration features, and cloud-native CI/CD automation capabilities. The decision to adopt either platform should be based on organizational priorities, such as whether the team values enterprise governance and compliance (Azure DevOps) or agility, flexibility, and community-driven development (GitHub).

10. Conclusions

In conclusion, there is no universal answer to whether Azure DevOps or GitHub is the better CI/CD platform—it depends entirely on the organization's software development strategy, security needs, scalability requirements, and budget constraints. Companies operating in highly regulated industries that require enterprise-scale security and structured DevOps governance will find Azure DevOps to be the superior choice. Software teams prioritizing agile development, Git-native workflows, and open-source collaboration will benefit from GitHub's intuitive platform, automation capabilities, and seamless developer experience. As CI/CD tools continue to evolve, the decision to use Azure DevOps, GitHub, or a hybrid of both should be driven by an organization's long-term DevOps objectives, infrastructure strategy, and security posture, ensuring a future-proof software development pipeline that meets both current and emerging business needs.

Author Contributions: V.M., D.G. and N.H. were involved in the entire process of producing this paper, including conceptualization, methodology, modeling, validation, visualization, and manuscript preparation. All authors have read and agreed to the published version of the manuscript.

Funding: This work has been accomplished with financial support from the European Regional Development Fund within the Operational Program “Bulgarian National Recovery and Resilience Plan”, the procedure for the direct provision of grants “Establishing a network of research higher education institutions in Bulgaria”, and under Project BG-RRP-2.004-0005 “Improving the research capacity and quality to achieve international recognition and resilience of TU-Sofia (IDEAS)”.

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare that they have no conflicts of interest.

References

1. Zaal, S. *Azure DevOps Explained*; Packt Publishing: Birmingham, UK, 2021.
2. Sinha, C. *Mastering Azure DevOps: A Comprehensive Guide to Implementing CI/CD Pipelines*; Apress: New York, NY, USA, 2021.
3. Karthik, A. *Azure DevOps for Web Developers*; Packt Publishing: Birmingham, UK, 2020.
4. Been, H. *Azure DevOps Server 2022: Implementing DevOps Using Azure DevOps Server*; Packt Publishing: Birmingham, UK, 2022.
5. Soni, M. *DevOps with Azure: Implementing DevOps Using Microsoft Azure*; Apress: New York, NY, USA, 2020.
6. Modi, R. *Deep-Dive Terraform on Azure: Automated Delivery and Deployment of Azure Solutions*; Apress: New York, NY, USA, 2021.
7. Price, M.J. *Azure DevOps for Beginners: A Step-by-Step Guide to CI/CD Pipelines*; Packt Publishing: Birmingham, UK, 2022.
8. Mitesh, S. *Hands-On Azure DevOps: Implementing CI/CD Pipelines*; Apress: New York, NY, USA, 2021.
9. ISO 27001; ISO-International Organization for Standardization. ISO: Geneva, Switzerland, 2022. Available online: <https://www.iso.org/home.html> (accessed on 6 March 2025).
10. SOC 2, Systems and Organization Controls. Available online: <https://soc2.co.uk/> (accessed on 6 March 2025).
11. GDPR, EUR-Lex-European Union law. Available online: <https://eur-lex.europa.eu/> (accessed on 6 March 2025).
12. HIPAA, U.S. Department of Labor. Available online: <https://www.dol.gov/> (accessed on 6 March 2025).
13. FedRAMP, Federal Risk and Authorization Management Program. Available online: <https://www.fedramp.gov/> (accessed on 6 March 2025).
14. Karthik, A. *Azure DevOps for .NET Developers*; Packt Publishing: Birmingham, UK, 2021.
15. Zaal, S. *Azure DevOps: A Complete Guide to CI/CD Pipelines*; Packt Publishing: Birmingham, UK, 2022.
16. Chacon, S.; Straub, B. *Pro Git*; Apress: New York, NY, USA, 2014.
17. Pipinellis, A. *GitHub Essentials: Unleash the Power of Collaborative Development*; Packt Publishing: Birmingham, UK, 2018.
18. Soni, M. *Mastering GitHub Actions: Automate Your Workflow with CI/CD*; Packt Publishing: Birmingham, UK, 2021.
19. Westby, E.J.H. *Git for Teams: A User-Centered Approach to Creating Efficient Workflows in Git*; O'Reilly Media: Sebastopol, CA, USA, 2015.
20. Loeliger, J.; McCullough, M. *Version Control with Git: Powerful Tools and Techniques for Collaborative Software Development*; O'Reilly Media: Sebastopol, CA, USA, 2012.
21. Laster, B. *Learning GitHub Actions: Automation and Integration of CI/CD with GitHub*; O'Reilly Media: Sebastopol, CA, USA, 2023.
22. Silverman, R.E. *Git Pocket Guide: A Working Introduction*; O'Reilly Media: Sebastopol, CA, USA, 2013.
23. Reddington, C. *GitHub Actions in Action*; Manning: Shelter Island, NY, USA, 2022.
24. Yuen, B.; Matyushentsev, A.; Ekenstam, T.; Suen, J. *GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux*; O'Reilly Media: Sebastopol, CA, USA, 2021.
25. Forsgren, N.; Humble, J.; Kim, G. *Accelerate: The Science of Lean Software and DevOps*; IT Revolution Press: Portland, OR, USA, 2018.
26. Kim, G.; Humble, J.; Debois, P.; Willis, J. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*; IT Revolution Press: Portland, OR, USA, 2021.
27. Humble, J.; Farley, D. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*; Addison-Wesley: White Plains, NY, USA, 2010.
28. Davis, J.; Daniels, R. *Effective DevOps: Building a Culture of Collaboration, Affinity, and Tooling at Scale*; O'Reilly Media: Sebastopol, CA, USA, 2016.
29. Beyer, B.; Jones, C.; Petoff, J.; Murphy, N.R. *Site Reliability Engineering: How Google Runs Production Systems*; O'Reilly Media: Sebastopol, CA, USA, 2016.
30. Morris, K. *Infrastructure as Code: Managing Servers in the Cloud*; O'Reilly Media: Sebastopol, CA, USA, 2016.
31. Duvall, P.M.; Matyas, S.; Glover, A. *Continuous Integration: Improving Software Quality and Reducing Risk*; Addison-Wesley: White Plains, NY, USA, 2007.
32. Kim, G.; Behr, K.; Spafford, G. *The Phoenix Project: A Novel About IT, DevOps, and Helping Your Business Win*; IT Revolution Press: Portland, OR, USA, 2018.
33. Humble, J.; Molesky, J.; O'Reilly, B. *Lean Enterprise: How High-Performance Organizations Innovate at Scale*; O'Reilly Media: Sebastopol, CA, USA, 2015.
34. Krief, M. *Terraform Cookbook—Second Edition: Provision, Run, and Scale Cloud Architecture with Real-World Examples Using Terraform*; Packt Publishing: Birmingham, UK, 2023.
35. Brikman, Y. *Terraform: Up and Running: Writing Infrastructure as Code*, 3rd ed.; O'Reilly Media: Sebastopol, CA, USA, 2022.
36. Winkler, S. *Terraform in Action*, 1st ed.; Manning: Shelter Island, NY, USA, 2021.
37. Moyle, E.; Kelley, D. *Practical Cybersecurity Architecture—Second Edition: A Guide to Creating and Implementing Robust Designs for Cybersecurity Architects*; Packt Publishing: Birmingham, UK, 2023.
38. Hüttermann, M. *DevOps for Developers*; Apress: New York, NY, USA, 2012.
39. Newman, S. *Building Microservices: Designing Fine-Grained Systems*, 2nd ed.; O'Reilly Media: Sebastopol, CA, USA, 2021.

40. Arundel, J.; Domingus, J. *Cloud Native DevOps with Kubernetes: Building, Deploying, and Scaling Modern Applications in the Cloud*, 2nd ed.; O'Reilly Media: Sebastopol, CA, USA, 2022.
41. Farcic, V.; Pope, D. *DevOps Toolkit Series*; Leanpub: Victoria, BC, Canada, 2020.
42. Ritchie, R. *Continuous Delivery with Docker and Jenkins: Create Secure Applications by Building Complete CI/CD Pipelines*, 2nd ed.; Packt Publishing: Birmingham, UK, 2019.
43. Mueller, E.; Wickett, J.; Karayanev, P. *The Practice of Cloud System Administration: DevOps and SRE Practices for Web Services*; Addison-Wesley: White Plains, NY, USA, 2021.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.